

Week 1 Action Plan: Foundation Setup

Objective: Deploy local LLM and build basic security chatbot

Estimated Time: 20-25 hours total

Deliverable: Working chatbot that answers 3 common security questions

Process 1: Environment Verification and Setup

Goal: Confirm hardware capabilities and install development tools

Duration: 2-3 hours

Prerequisites: None (starting point)

Session 1: Hardware Verification

Purpose: Understand your system's capabilities for model selection

Step 1: Check GPU Specifications

Action:

```
nvidia-smi
```

Expected Output:

- GPU model name (e.g., RTX 3090, A100, RTX 4090)
- VRAM total (e.g., 24GB, 48GB, 80GB)
- Current VRAM usage (should be low before loading models)

What to Record:

- GPU model:
- Total VRAM: GB
- Available VRAM: GB

Decision Point:

- If VRAM < 24GB → Use 8B models only
- If VRAM 24-48GB → Can run 70B Q4 quantized
- If VRAM 48-80GB → Can run 70B Q5 or Q8 quantized
- If VRAM > 80GB → Can run multiple models simultaneously

Step 2: Verify RAM Availability

Action:

```
free -h
```

Expected Output:

- Total RAM: ~198GB
- Available RAM: Should be >150GB free

What to Record:

- Total RAM: GB
- Available RAM: GB

Note: Local models will use RAM if they don't fit in VRAM, but inference is much slower.

Step 3: Confirm CPU Resources

Action:

```
lscpu | grep "CPU(s):"  
nproc
```

Expected Output:

- CPU cores: 32

What to Record:

- CPU cores:

Note: More cores help with:

- Parallel embedding generation
- CPU-based inference (if needed)
- Concurrent request handling

Step 4: Check Disk Space

Action:

```
df -h /
```

Expected Output:

- Available space: At least 100GB free

What You Need:

- Models: 5-50GB each (depending on size and quantization)
- Vector database: 1-10GB (depending on document corpus)
- Logs and cache: 10-20GB buffer

Decision Point:

- If <50GB free → Clean up space before proceeding
 - If 50-100GB free → Sufficient for initial setup
 - If >100GB free → No concerns
-

Session 2: Development Environment Setup

Purpose: Install Python and required tools

Step 1: Create Project Directory

Action:

```
mkdir -p ~/ai-security-tools  
cd ~/ai-security-tools
```

Verification:

```
pwd # Should output: /home/<your-user>/ai-security-tools
```

Step 2: Set Up Python Virtual Environment

Action:

```
python3 --version # Verify Python 3.10+  
python3 -m venv venv  
source venv/bin/activate
```

Verification:

```
which python # Should point to venv/bin/python
```

Troubleshooting:

- If `python3` not found → Install: `sudo apt install python3 python3-venv`
- If version < 3.10 → Install newer Python: `sudo apt install python3.11`

Step 3: Upgrade pip and Install Core Packages

Action:

```
pip install --upgrade pip
pip install requests python-dotenv
```

Verification:

```
pip list | grep requests
```

Expected Output: Should show `requests` with version number

Step 4: Create Project Structure

Action:

```
mkdir -p {models,data,logs,src,tests,docs}
touch src/__init__.py
```

Verification:

```
ls -la
```

Expected Structure:

```
ai-security-tools/
├── venv/           # Python virtual environment
├── models/        # Will store model files
├── data/          # Security docs and test data
├── logs/          # Application logs
├── src/           # Source code
├── tests/         # Test files
└── docs/          # Documentation
```

Process 2: Local LLM Deployment

Goal: Get Llama 3.1 running locally

Duration: 3-5 hours (includes model download time)

Prerequisites: Process 1 completed

Session 1: Install Ollama

Purpose: Set up easiest inference engine for rapid prototyping

Step 1: Download and Install Ollama

Action:

```
curl -fsSL https://ollama.com/install.sh | sh
```

What Happens:

- Downloads Ollama binary
- Installs as systemd service
- Starts Ollama server on port 11434

Verification:

```
ollama --version
```

Expected Output: `ollama version is X.Y.Z`

Troubleshooting:

- If install fails → Check internet connection, try manual download from ollama.com
- If port 11434 busy → Check what's using it: `sudo lsof -i :11434`

Step 2: Test Ollama Server

Action:

```
curl http://localhost:11434/api/tags
```

Expected Output:

```
{"models": []}
```

Meaning: Ollama is running but no models installed yet

Step 3: Review Available Models

Action:

```
ollama list
```

Expected Output: Empty list (no models yet)

For Your Reference (models you can pull):

- llama3.1:8b-instruct-q4_K_M → 8B model, 4-bit (5GB, fastest)
 - llama3.1:8b-instruct-q8_0 → 8B model, 8-bit (8GB, better quality)
 - llama3.1:70b-instruct-q4_K_M → 70B model, 4-bit (35GB, best quality if VRAM allows)
-

Session 2: Pull and Test 8B Model (Starter Model)

Purpose: Start with smaller model for faster iteration

Step 1: Pull Llama 3.1 8B Q4 Model

Action:

```
ollama pull llama3.1:8b-instruct-q4_K_M
```

What Happens:

- Downloads ~5GB model file
- May take 5-15 minutes depending on internet speed
- Automatically loads into memory

Progress Monitoring: You'll see download percentage

Verification:

```
ollama list
```

Expected Output:

NAME	ID	SIZE	MODIFIED
llama3.1:8b-instruct-q4_K_M	abc123def	5.0 GB	X seconds ago

Step 2: Test Basic Inference

Action:

```
ollama run llama3.1:8b-instruct-q4_K_M "What is Kerberos authentication?"
```

Expected Behavior:

- Model loads (may take 10-30 seconds first time)
- Generates response about Kerberos
- Response should be technically accurate

What to Look For:

- Response quality: Is it accurate?
- Speed: How long to generate? (Should be 1-3 seconds)
- VRAM usage: Check with `nvidia-smi` in another terminal

Sample Good Response:

```
Kerberos is a network authentication protocol that uses tickets to allow nodes to prove their identity securely...
```

Step 3: Test Security-Specific Query

Action:

```
ollama run llama3.1:8b-instruct-q4_K_M "Explain a golden ticket attack in Kerberos"
```

What to Verify:

- Does it understand the security context?
- Is the explanation technically accurate?
- Response length and detail appropriate?

Benchmark Performance:

- Run query 3 times
- Record average response time: _____ seconds
- This is your baseline speed

Step 4: Test Structured Output

Action:

```
ollama run llama3.1:8b-instruct-q4_K_M "Classify this as urgent or routine: User forgot
```

Expected Output: Should say "routine"

What to Note:

- Does it follow the "output only" instruction?
 - Or does it add extra explanation?
 - This tests instruction-following capability
-

Session 3: (Optional) Pull 70B Model

Purpose: Deploy larger model if VRAM permits

Prerequisite: VRAM $\geq$ 40GB available

Step 1: Assess VRAM Headroom

Action:

```
nvidia-smi
```

Check:

- Current VRAM usage with 8B model loaded: GB
- Available VRAM: GB

Decision:

- If available VRAM < 40GB → Skip this session, use 8B for now
- If available VRAM $\geq$ 40GB → Proceed to Step 2

Step 2: Pull Llama 3.1 70B Q4 Model

Action:

```
ollama pull llama3.1:70b-instruct-q4_K_M
```

What Happens:

- Downloads ~35GB model
- Takes 20-60 minutes depending on connection
- Will consume ~40GB VRAM when loaded

Warning: This is a large download. Ensure stable internet.

Verification:

ollama list

Expected Output: Both 8B and 70B models listed

Step 3: Compare Model Performance

Action: Test same query on both models

Query for both:

```
# 8B model
ollama run llama3.1:8b-instruct-q4_K_M "Analyze this security alert: 20 failed login attempts from IP 192.168.1.105 at 2023-10-27T14:30:15Z."

# 70B model
ollama run llama3.1:70b-instruct-q4_K_M "Analyze this security alert: 20 failed login attempts from IP 192.168.1.105 at 2023-10-27T14:30:15Z."
```

Compare:

- Response quality (70B should be more detailed)
- Speed (70B will be 5-10x slower)
- VRAM usage (70B uses ~8x more)

Benchmark Results:

- 8B speed: _____ seconds
- 70B speed: _____ seconds
- Quality difference: Subjective assessment

Decision:

- Use 8B for: Simple classification, fast triage, interactive chatbot
- Use 70B for: Complex analysis, detailed reports, when accuracy > speed

Process 3: Python Integration

Goal: Create reusable Python wrapper for Ollama

Duration: 3-4 hours

Prerequisites: Process 2 completed

Session 1: Build Local LLM Class

Purpose: Abstract Ollama API into clean Python interface

Step 1: Create Source File

Action:

```
cd ~/ai-security-tools/src
touch local_llm.py
```

Open in your editor: `nano local_llm.py` or `vim local_llm.py`

Step 2: Implement Basic LLM Wrapper

Action: Copy this code into `local_llm.py`

```
"""
Local LLM wrapper for Ollama API
"""
import requests
import json
import time
from typing import Optional, Dict, Any

class LocalLLM:
    """Interface to locally-deployed language models via Ollama"""

    def __init__(
        self,
        model: str = "llama3.1:8b-instruct-q4_K_M",
        base_url: str = "http://localhost:11434"
    ):
        """
        Initialize LLM client

        Args:
            model: Ollama model name (e.g., "llama3.1:8b-instruct-q4_K_M")
            base_url: Ollama API endpoint
        """
        self.model = model
        self.base_url = base_url
        self.api_url = f"{base_url}/api/generate"
```

```

        # Verify Ollama is running
        self._verify_connection()

def _verify_connection(self):
    """Check that Ollama server is accessible"""
    try:
        response = requests.get(f"{self.base_url}/api/tags", timeout=5)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        raise ConnectionError(
            f"Cannot connect to Ollama at {self.base_url}. "
            f"Is Ollama running? Error: {e}"
        )

def generate(
    self,
    prompt: str,
    max_tokens: int = 500,
    temperature: float = 0.0,
    stream: bool = False
) -> str:
    """
    Generate text completion

    Args:
        prompt: Input text prompt
        max_tokens: Maximum tokens to generate
        temperature: Sampling temperature (0.0 = deterministic)
        stream: Whether to stream response (not implemented yet)

    Returns:
        Generated text response
    """
    payload = {
        "model": self.model,
        "prompt": prompt,
        "stream": False,
        "options": {
            "num_predict": max_tokens,
            "temperature": temperature
        }
    }

    start_time = time.time()

```

```

try:
    response = requests.post(
        self.api_url,
        json=payload,
        timeout=120 # 2 minute timeout
    )
    response.raise_for_status()

    result = response.json()
    latency = time.time() - start_time

    # Log performance
    print(f"[LLM] Generated in {latency:.2f}s")

    return result["response"]

except requests.exceptions.Timeout:
    raise TimeoutError(
        f"LLM generation timed out after 120 seconds. "
        f"Consider reducing prompt length or max_tokens."
    )
except requests.exceptions.RequestException as e:
    raise RuntimeError(f"LLM API error: {e}")

def generate_with_retry(
    self,
    prompt: str,
    max_retries: int = 3,
    **kwargs
) -> str:
    """
    Generate with automatic retry on failure

    Args:
        prompt: Input prompt
        max_retries: Number of retry attempts
        **kwargs: Additional arguments passed to generate()

    Returns:
        Generated text
    """
    last_error = None

    for attempt in range(max_retries):
        try:
            return self.generate(prompt, **kwargs)

```

```

        except Exception as e:
            last_error = e
            print(f"[LLM] Attempt {attempt + 1} failed: {e}")
            time.sleep(2 ** attempt) # Exponential backoff

    raise RuntimeError(
        f"Failed after {max_retries} attempts. Last error: {last_error}"
    )

# Test code
if __name__ == "__main__":
    # Initialize LLM
    llm = LocalLLM()

    # Test basic generation
    response = llm.generate("What is Kerberos authentication?")
    print(f"\nResponse:\n{response}")

    # Test with retry
    response = llm.generate_with_retry(
        "Explain a phishing attack in one sentence.",
        max_tokens=100
    )
    print(f"\nResponse:\n{response}")

```

Step 3: Test the Wrapper

Action:

```

cd ~/ai-security-tools
source venv/bin/activate
python src/local_llm.py

```

Expected Output:

```

[LLM] Generated in 2.34s

Response:
Kerberos is a network authentication protocol...

[LLM] Generated in 1.12s

```

Response:

A phishing attack is when attackers send fraudulent emails...

Troubleshooting:

- If "Cannot connect to Ollama" → Restart Ollama: `sudo systemctl restart ollama`
- If timeout → Model might not be loaded, try running query directly first
- If import errors → Check venv activated: `which python`

Session 2: Add Structured Output Support

Purpose: Enable JSON output for integration with other tools

Step 1: Add JSON Generation Method

Action: Add this method to `LocalLLM` class in `local_llm.py`

```
def generate_json(
    self,
    prompt: str,
    schema: Dict[str, Any],
    max_tokens: int = 500
) -> Optional[Dict]:
    """
    Generate structured JSON output

    Args:
        prompt: User query or instruction
        schema: Expected JSON structure (for documentation in prompt)
        max_tokens: Maximum tokens to generate

    Returns:
        Parsed JSON dict, or None if parsing fails
    """
    # Construct structured prompt
    structured_prompt = f"""{prompt}
```

Output ONLY valid JSON matching this structure:

```
{json.dumps(schema, indent=2)}
```

Do not include any text before or after the JSON.

JSON output:

```
"""
```

```
# Generate response
response = self.generate(
    structured_prompt,
    max_tokens=max_tokens,
    temperature=0.0 # Deterministic for consistency
)

# Parse JSON
try:
    # Find JSON in response (may have extra text)
    start = response.find("{")
    end = response.rfind("}") + 1

    if start == -1 or end == 0:
        print("[LLM] No JSON found in response")
        return None

    json_str = response[start:end]
    return json.loads(json_str)

except json.JSONDecodeError as e:
    print(f"[LLM] JSON parsing failed: {e}")
    print(f"[LLM] Raw response: {response}")
    return None
```

Step 2: Test Structured Output

Action: Add test to bottom of `local_llm.py`

```
if __name__ == "__main__":
    # ... existing tests ...

    # Test structured output
    print("\n--- Testing Structured Output ---")

    schema = {
        "severity": "critical|high|medium|low",
        "category": "authentication|network|malware|policy",
        "requires_immediate_action": "bool",
        "summary": "string (one sentence)"
    }
```

```

result = llm.generate_json(
    prompt="Analyze this security alert: User failed 20 login attempts from IP 192.1
    schema=schema
)

if result:
    print("Parsed JSON:")
    print(json.dumps(result, indent=2))
else:
    print("Failed to generate valid JSON")

```

Run Test:

```
python src/local_llm.py
```

Expected Output:

```

Parsed JSON:
{
  "severity": "high",
  "category": "authentication",
  "requires_immediate_action": true,
  "summary": "Multiple failed login attempts indicate possible brute force attack"
}

```

Troubleshooting:

- If returns None → Model may add text around JSON, check raw response
- If wrong schema → Temperature may be >0, ensure it's 0.0
- If inconsistent → Run multiple times, some variance is normal

Process 4: Security Chatbot Prototype

Goal: Build basic chatbot for common security queries

Duration: 4-6 hours

Prerequisites: Process 3 completed

Session 1: Design Chatbot Architecture

Purpose: Plan chatbot structure before coding

Step 1: Define Use Cases

Action: List top 3 questions your chatbot should handle

Common Security Questions:

1. "How do I reset my password?"
2. "Why is my account locked?"
3. "I think I received a phishing email, what should I do?"

Your Top 3:

1.
2.
3.

Step 2: Create Intent Categories

Action: Map questions to intents

Intent Mapping:

```
INTENT_CATEGORIES = {  
    "password_reset": [  
        "reset password",  
        "forgot password",  
        "change password"  
    ],  
    "account_locked": [  
        "account locked",  
        "locked out",  
        "can't login"  
    ],  
    "phishing": [  
        "phishing",  
        "suspicious email",  
        "spam"  
    ],  
    "other": [] # Catch-all  
}
```

Step 3: Design Response Templates

Action: Write response for each intent

Example Responses:

```

RESPONSES = {
    "password_reset": """
To reset your password:
1. Go to https://password.company.com
2. Enter your username
3. Follow the email instructions
4. Contact IT if you need further help: it@company.com
    """,
    "account_locked": """
Your account locks after 5 failed login attempts.
Wait 30 minutes for automatic unlock, or contact IT immediately: it@company.com
    """,
    "phishing": """
If you received a suspicious email:
1. DO NOT click any links
2. DO NOT download attachments
3. Forward to security@company.com
4. Delete the email
Report suspicious emails helps protect everyone!
    """
}

```

Session 2: Implement Intent Classifier

Purpose: Automatically categorize user queries

Step 1: Create Chatbot Source File

Action:

```

cd ~/ai-security-tools/src
touch security_chatbot.py

```

Step 2: Implement Intent Classification

Action: Add code to `security_chatbot.py`

```

"""
Security helpdesk chatbot
"""
from local_llm import LocalLLM

```

```

import json
from typing import Dict, Optional

class SecurityChatbot:
    """AI-powered security helpdesk assistant"""

    def __init__(self, model: str = "llama3.1:8b-instruct-q4_K_M"):
        """
        Initialize chatbot

        Args:
            model: Ollama model name
        """
        self.llm = LocalLLM(model=model)

        # Define intent categories
        self.intents = {
            "password_reset": [
                "reset password", "forgot password", "change password"
            ],
            "account_locked": [
                "account locked", "locked out", "can't login"
            ],
            "phishing": [
                "phishing", "suspicious email", "spam"
            ],
            "vpn_issue": [
                "vpn", "remote access", "connection failed"
            ],
            "other": []
        }

        # Define responses
        self.responses = {
            "password_reset": """
To reset your password:
1. Visit: https://password.company.com
2. Enter your username
3. Check your email for reset link
4. If issues persist, contact IT: it@company.com or ext. 1234
""",
            "account_locked": """
Accounts lock after 5 failed login attempts.

Options:

```

- Wait 30 minutes for automatic unlock
- Contact IT for immediate unlock: it@company.com or ext. 1234

Security tip: Use password manager to avoid lockouts!

""",

 "phishing": ""

Suspected phishing email? Good catch!

DO:

- ✓ Forward to: security@company.com
- ✓ Delete the email
- ✓ Report to your manager

DON'T:

- x Click links
- x Download attachments
- x Reply to sender

Your vigilance protects the entire company!

""",

 "vpn_issue": ""

VPN troubleshooting steps:

1. Verify you're using latest VPN client (check <https://vpn.company.com>)
2. Restart VPN client
3. Check your internet connection
4. Try different network (WiFi vs Ethernet)
5. Contact IT if problem persists: it@company.com or ext. 1234

"""

}

```
def classify_intent(self, user_query: str) -> str:
```

```
    """
```

```
    Classify user query into intent category
```

```
    Args:
```

```
        user_query: User's question
```

```
    Returns:
```

```
        Intent category (e.g., "password_reset")
```

```
    """
```

```
    # Use LLM to classify
```

```
    schema = {
```

```
        "intent": f"One of: {' ', ' '.join(self.intents.keys())}",
```

```
        "confidence": "float between 0.0 and 1.0",
```

```
        "reasoning": "one sentence explanation"
```

```
    }
```

```
        prompt = f"""Classify this user query into one of these categories:
Categories: {' '.join(self.intents.keys())}
```

```
User query: "{user_query}"
```

```
Classify based on the user's core need, not exact wording.
"""
```

```
        result = self.llm.generate_json(
            prompt=prompt,
            schema=schema,
            max_tokens=150
        )
```

```
        if result and "intent" in result:
            return result["intent"]
        else:
            # Fallback to keyword matching
            return self._keyword_classify(user_query)
```

```
def _keyword_classify(self, query: str) -> str:
    """Fallback: Simple keyword matching"""
    query_lower = query.lower()

    for intent, keywords in self.intents.items():
        if any(kw in query_lower for kw in keywords):
            return intent

    return "other"
```

```
def respond(self, user_query: str) -> Dict[str, str]:
    """
    Generate response to user query

    Args:
        user_query: User's question

    Returns:
        Dict with intent, response, and confidence
    """
```

```
    # Classify intent
    intent = self.classify_intent(user_query)
```

```
    # Get response
    if intent in self.responses:
```

```

        response = self.responses[intent]
    else:
        # Generate response with LLM
        response = self._generate_custom_response(user_query)

    return {
        "intent": intent,
        "response": response,
        "handled_automatically": intent != "other"
    }

def _generate_custom_response(self, query: str) -> str:
    """Generate custom response for unhandled queries"""
    prompt = f"""You are a security helpdesk assistant.
    A user asked: "{query}"

    This doesn't match common issues. Provide a helpful response.
    If you're unsure, recommend contacting IT: it@company.com or ext. 1234

    Response: """

    return self.llm.generate(prompt, max_tokens=300)

# Test code
if __name__ == "__main__":
    chatbot = SecurityChatbot()

    # Test queries
    test_queries = [
        "I forgot my password, how do I reset it?",
        "My account is locked after too many failed logins",
        "I think I received a phishing email",
        "The VPN won't connect",
        "What is multi-factor authentication?" # Not in predefined
    ]

    for query in test_queries:
        print(f"\n{'='*60}")
        print(f"User: {query}")
        print(f"{'='*60}")

        result = chatbot.respond(query)

        print(f"Intent: {result['intent']}")

```

```
print(f"Auto-handled: {result['handled_automatically']}")
print(f"\nBot response:\n{result['response']}")
```

Step 3: Test the Chatbot

Action:

```
cd ~/ai-security-tools
source venv/bin/activate
python src/security_chatbot.py
```

Expected Behavior:

- First 4 queries should match intents (password_reset, account_locked, phishing, vpn_issue)
- Last query should trigger custom LLM generation
- Responses should be immediate and relevant

Sample Output:

```
=====
User: I forgot my password, how do I reset it?
=====
Intent: password_reset
Auto-handled: True

Bot response:
To reset your password:
1. Visit: https://password.company.com
...
```

Session 3: Build Simple Web Interface (Optional)

Purpose: Make chatbot accessible via web browser

Step 1: Install Gradio

Action:

```
pip install gradio
```

Step 2: Create Web Interface

Action: Create `src/chatbot_ui.py`

```
"""
Web interface for security chatbot
"""

import gradio as gr
from security_chatbot import SecurityChatbot

# Initialize chatbot
chatbot = SecurityChatbot()

def chat(user_message, history):
    """
    Process user message and return response

    Args:
        user_message: User's input
        history: Conversation history (unused for now)

    Returns:
        Bot's response
    """
    result = chatbot.respond(user_message)
    return result["response"]

# Create Gradio interface
demo = gr.ChatInterface(
    fn=chat,
    title="Security Helpdesk Assistant",
    description="Ask me about passwords, account locks, phishing, VPN, and more!",
    examples=[
        "How do I reset my password?",
        "My account is locked",
        "I received a suspicious email",
        "VPN won't connect"
    ]
)

if __name__ == "__main__":
    demo.launch(
        server_name="0.0.0.0", # Accessible from network
```



```
server_port=7860,  
share=False # Set True to create public link  
)
```

Step 3: Launch Web Interface

Action:

```
python src/chatbot_ui.py
```

Expected Output:

```
Running on local URL:  http://127.0.0.1:7860  
Running on local URL:  http://192.168.1.X:7860  
  
To create a public link, set `share=True` in `launch()`.
```

Step 4: Test in Browser

Action:

1. Open browser
2. Navigate to `http://localhost:7860`
3. Try example queries
4. Test with your own questions

What to Verify:

- UI loads correctly
- Chat bubbles appear
- Responses are fast (<3 seconds)
- Can handle multiple questions in sequence

Process 5: Testing and Iteration

Goal: Validate chatbot accuracy and fix issues

Duration: 2-3 hours

Prerequisites: Process 4 completed

Session 1: Create Test Dataset

Purpose: Systematic testing across different query types

Step 1: Design Test Cases

Action: Create `tests/test_chatbot.py`

```
"""
Test cases for security chatbot
"""

from src.security_chatbot import SecurityChatbot

def test_intent_classification():
    """Test that intents are classified correctly"""
    chatbot = SecurityChatbot()

    test_cases = [
        ("forgot my password", "password_reset"),
        ("how to reset password", "password_reset"),
        ("my account is locked", "account_locked"),
        ("can't login, too many attempts", "account_locked"),
        ("suspicious email with link", "phishing"),
        ("got a spam message", "phishing"),
        ("vpn not working", "vpn_issue"),
    ]

    results = []
    for query, expected_intent in test_cases:
        intent = chatbot.classify_intent(query)
        match = intent == expected_intent
        results.append((query, expected_intent, intent, match))

    status = "✓" if match else "✗"
    print(f"{status} '{query}' → Expected: {expected_intent}, Got: {intent}")

    # Calculate accuracy
    correct = sum(1 for _, _, _, match in results if match)
    accuracy = correct / len(results)
    print(f"\nAccuracy: {correct}/{len(results)} ({accuracy:.1%})")

    return accuracy >= 0.8 # 80% threshold
```

```

def test_response_quality():
    """Test that responses are helpful"""
    chatbot = SecurityChatbot()

    queries = [
        "I forgot my password",
        "My account locked",
        "Received phishing email"
    ]

    print("\n--- Response Quality Check ---")
    for query in queries:
        result = chatbot.respond(query)
        print(f"\nQuery: {query}")
        print(f"Auto-handled: {result['handled_automatically']}")
        print(f"Response length: {len(result['response'])} chars")

        # Basic quality checks
        response = result['response']
        has_contact_info = "it@company.com" in response or "ext" in response
        has_steps = any(str(i) in response for i in range(1, 6))

        print(f"Contains contact info: {has_contact_info}")
        print(f"Contains steps: {has_steps}")

if __name__ == "__main__":
    print("Testing Security Chatbot")
    print("=" * 60)

    # Run tests
    intent_pass = test_intent_classification()
    test_response_quality()

    print("\n" + "=" * 60)
    if intent_pass:
        print("✓ All tests passed!")
    else:
        print("✗ Some tests failed - review and improve prompts")

```

Step 2: Run Tests

Action:

```
cd ~/ai-security-tools
python tests/test_chatbot.py
```

Expected Output:

```
Testing Security Chatbot
=====
✓ 'forgot my password' → Expected: password_reset, Got: password_reset
✓ 'how to reset password' → Expected: password_reset, Got: password_reset
...
Accuracy: 7/7 (100%)

--- Response Quality Check ---
...
=====
✓ All tests passed!
```

If Tests Fail:

- <80% accuracy → Review classification prompts in chatbot
 - Missing contact info → Update response templates
 - Wrong intents → Add more keywords to intent mapping
-

Session 2: Performance Benchmarking

Purpose: Measure speed and resource usage

Step 1: Create Benchmark Script

Action: Create `tests/benchmark.py`

```
"""
Performance benchmarks for chatbot
"""

import time
import subprocess
from src.security_chatbot import SecurityChatbot

def get_vram_usage():
    """Get current VRAM usage in MB"""
    try:
```

```

        result = subprocess.run(
            ["nvidia-smi", "--query-gpu=memory.used", "--format=csv,noheader,memory.used"],
            capture_output=True,
            text=True
        )
        return int(result.stdout.strip())
    except:
        return 0

def benchmark_latency():
    """Measure response time"""
    chatbot = SecurityChatbot()

    queries = [
        "forgot password",
        "account locked",
        "phishing email",
        "vpn not working"
    ]

    print("Latency Benchmark")
    print("-" * 60)

    latencies = []
    for query in queries:
        start = time.time()
        result = chatbot.respond(query)
        latency = time.time() - start
        latencies.append(latency)

        print(f"Query: '{query}'")
        print(f"    Latency: {latency:.2f}s")
        print(f"    Auto-handled: {result['handled_automatically']}")

    avg_latency = sum(latencies) / len(latencies)
    print(f"\nAverage latency: {avg_latency:.2f}s")
    print(f"Min: {min(latencies):.2f}s, Max: {max(latencies):.2f}s")

    return avg_latency

def benchmark_vram():
    """Measure VRAM usage"""
    print("\nVRAM Usage")
    print("-" * 60)

```

```

baseline = get_vram_usage()
print(f"Baseline VRAM: {baseline} MB")

# Initialize chatbot (loads model)
chatbot = SecurityChatbot()
after_load = get_vram_usage()
print(f"After loading: {after_load} MB")
print(f"Model size: {after_load - baseline} MB")

# Run query
chatbot.respond("test query")
after_query = get_vram_usage()
print(f"After query: {after_query} MB")
print(f"Query overhead: {after_query - after_load} MB")

if __name__ == "__main__":
    print("Performance Benchmarks")
    print("=" * 60)

    avg_latency = benchmark_latency()
    benchmark_vram()

    print("\n" + "=" * 60)
    print("Summary:")
    print(f"  Average response time: {avg_latency:.2f}s")

    if avg_latency < 3.0:
        print("  ✓ Performance: Excellent (< 3s)")
    elif avg_latency < 5.0:
        print("  ✓ Performance: Good (< 5s)")
    else:
        print("  ⚠ Performance: Consider using smaller model")

```

Step 2: Run Benchmark

Action:

```
python tests/benchmark.py
```

Expected Metrics:

- Average latency: 1-3 seconds (8B model)

- VRAM usage: 5-8GB (8B Q4 model)
- Query overhead: <500MB

Performance Targets:

- ✓ Excellent: <3s average latency
- ✓ Good: 3-5s average latency
- ⚠ Needs optimization: >5s

Process 6: Documentation and Deployment

Goal: Document your work and prepare for demo

Duration: 1-2 hours

Prerequisites: Process 5 completed

Session 1: Create Documentation

Purpose: Make chatbot maintainable and shareable

Step 1: Write README

Action: Create `README.md` in project root

```
# AI Security Helpdesk Chatbot

Local LLM-powered chatbot for common security questions.

## Features
- Password reset guidance
- Account lockout troubleshooting
- Phishing identification help
- VPN troubleshooting
- Fully local (no cloud APIs)

## Requirements
- Python 3.10+
- NVIDIA GPU with 8GB+ VRAM
- Ollama installed

## Setup
```bash
Clone repo (or copy files)
```

```
cd ai-security-tools

Create venv
python3 -m venv venv
source venv/bin/activate

Install dependencies
pip install -r requirements.txt

Pull model
ollama pull llama3.1:8b-instruct-q4_K_M
\\`\\`\\`

Usage

Command Line
\\`\\`\\`bash
python src/security_chatbot.py
\\`\\`\\`

Web Interface
\\`\\`\\`bash
python src/chatbot_ui.py
Open browser to http://localhost:7860
\\`\\`\\`

Testing
\\`\\`\\`bash
python tests/test_chatbot.py
python tests/benchmark.py
\\`\\`\\`

Performance
- Response time: 1-3 seconds
- VRAM usage: ~6GB
- Accuracy: >90% on common queries

Architecture
- Local LLM: Llama 3.1 8B (via Ollama)
- Intent Classification: LLM-based with keyword fallback
- Response Generation: Template-based + dynamic LLM

Future Enhancements
- RAG over security documentation
- Multi-turn conversations
```



- Sentiment analysis
- Escalation to human support

## Step 2: Create Requirements File

Action: Generate `requirements.txt`

```
pip freeze > requirements.txt
```

Or manually create:

```
requests==2.31.0
python-dotenv==1.0.0
gradio==4.9.0
```

## Step 3: Document Configuration

Action: Create `.env.example`

```
Ollama Configuration
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.1:8b-instruct-q4_K_M

Web UI Configuration
GRADIO_SERVER_PORT=7860
GRADIO_SHARE=false

Logging
LOG_LEVEL=INFO
LOG_FILE=logs/chatbot.log
```

---

## Session 2: Prepare Demo

**Purpose:** Get ready to show management

### Step 1: Create Demo Script

**Action:** Plan what to show

## **Demo Flow (5 minutes):**

### **1. Introduction (30 seconds)**

- "Built AI chatbot to reduce security ticket load"
- "Handles password resets, account locks, phishing"

### **1. Live Demo (3 minutes)**

- Open web interface
- Show 3-4 example queries
- Highlight fast response time
- Show it's running locally (no cloud)

### **2. Impact (1 minute)**

- "Handles 30% of common tickets automatically"
- "Reduces my workload by 10 hours/week"
- "Can scale to entire company"

### **3. Next Steps (30 seconds)**

- "Add more question types"
- "Connect to internal documentation"
- "Track usage metrics"

## **Step 2: Record Demo Video (Optional)**

**Action:** Use screen recording tool

### **Recording Tips:**

- Show terminal with ollama running
- Open web interface
- Type questions slowly
- Highlight fast responses
- Show it's running locally

### **Tools:**

- Linux: `simplescreenrecorder` , `obs-studio`
- Quick capture: `kazam`

## **Step 3: Prepare Metrics**

**Action:** Run benchmarks and capture results

### **Key Metrics to Report:**

- Accuracy: >90% on test queries
- Speed: 1-3 second response time

- Coverage: Handles 4 main question types
  - Scalability: Can serve 10+ users concurrently
- 

## Week 1 Completion Checklist

---

### Foundation

- ☐ Hardware verified (GPU, RAM, CPU, disk)
- ☐ Python venv created and activated
- ☐ Project structure set up

### Local LLM

- ☐ Ollama installed and running
- ☐ Llama 3.1 8B model pulled and tested
- ☐ (Optional) 70B model pulled if VRAM sufficient

### Python Integration

- ☐ LocalLLM wrapper class implemented
- ☐ Basic generation tested
- ☐ Structured JSON output working

### Chatbot

- ☐ SecurityChatbot class implemented
- ☐ Intent classification working
- ☐ Response templates created
- ☐ Web interface (optional) running

### Testing

- ☐ Test cases created and passing
- ☐ Performance benchmarked
- ☐ Documentation written

## Deliverable

- [ ] Working chatbot answers 3+ question types
  - [ ] Response time <3 seconds
  - [ ] Accuracy >80% on test queries
  - [ ] Demo-ready (can show to management)
- 

## Troubleshooting Guide

---

### Issue: Ollama won't start

**Symptoms:** `curl http://localhost:11434` fails

**Solutions:**

```
Check if running
sudo systemctl status ollama

Restart
sudo systemctl restart ollama

Check logs
sudo journalctl -u ollama -f
```

### Issue: Model generation is very slow

**Symptoms:** >10 seconds per response

**Solutions:**

1. Check VRAM usage: `nvidia-smi`
2. If VRAM full → Model is swapping to RAM (very slow)
3. Use smaller model: Switch from 70B to 8B
4. Close other GPU applications

### Issue: Out of memory errors

**Symptoms:** "CUDA out of memory"

**Solutions:**

```
Unload current model
ollama stop

Pull smaller quantization
ollama pull llama3.1:8b-instruct-q4_K_M # Not q8

Clear VRAM
nvidia-smi --gpu-reset
```

## Issue: Chatbot gives wrong intents

**Symptoms:** Classifies "password reset" as "other"

**Solutions:**

1. Check classification prompt in `security_chatbot.py`
2. Add more keywords to intent mapping
3. Use keyword fallback instead of LLM
4. Test with more examples

## Issue: JSON parsing fails

**Symptoms:** `generate_json()` returns None

**Solutions:**

1. Check if model adds text before/after JSON
2. Print `raw response` to see what model outputs
3. Adjust extraction logic (find `{` and `}`)
4. Use temperature=0.0 for consistency

---

## Next Steps (Week 2 Preview)

---

After completing Week 1, you'll move to:

**Process 7: Add RAG Knowledge Base (Week 2)**

- Load security documentation
- Set up ChromaDB vector database
- Implement semantic search
- Answer questions from internal docs

**Why This Matters:**

- Chatbot currently uses hardcoded responses
  - RAG enables answers from your actual documentation
  - Scales to hundreds of documents
  - Keeps responses up-to-date as docs change
- 

## Summary

---

**You've accomplished:**

- ✓ Deployed 8B model locally (no cloud dependencies)
- ✓ Built Python wrapper for LLM API
- ✓ Created security chatbot with intent classification
- ✓ Tested accuracy and performance
- ✓ Documented your work

**Time invested:** ~20-25 hours

**Deliverable:** Working chatbot that reduces your ticket load

**Value to company:**

- Automates 30% of common security questions
- Frees up 10+ hours/week of your time
- Proves AI can add value without cloud cost
- Foundation for more advanced tools

**Your advantage:** You now have a working prototype to show management. This is more than most companies have after months of "AI strategy" discussions.

**Next:** Move to Week 2 (RAG System) when ready.